PRA-vLLM: Separating Query-Addressed Logical Memory from Paged KV Placement

Jorge Simão
EInnovator R&D

August 2026

Abstract

vLLM virtualizes where sequential key–value (KV) blocks reside. Progressive Retrieval Attention (PRA) virtualizes a different choice: which subset of a logically large, non-prefix memory is active for a query. We define a capability-honest integration boundary between these mappings, implement a stable logical PRA block namespace and residency state machine, and implement a vLLM-Metal store that places selected K/V in native paged-cache blocks. On an Apple M5 with vLLM 0.28.0 and the vLLM-Metal plugin, a 25-request smoke study separates exact-prefix reuse from selected-memory recovery. A selected 365-token prompt and the 1,577-token full context both recover the codeword in 5/5 requests; the selected condition uses 76.9% fewer visible tokens and reduces cold time to first token (TTFT) from 776.6 to 80.7 ms. Warm TTFT is 46.2 versus 52.9 ms, while vLLM reports an aggregate prefix-cache hit rate of 75.3–83.4%. These measurements establish a strong black-box baseline. Separately, 80 controlled kernel measurements over five seeds, 16–512 selected tokens, and concurrency 1–8 match dense attention within maximum absolute error 0.00391. Warm native paged attention rises from 0.489 to 0.642 ms; immutable block sharing avoids up to 14.0 MB at concurrency eight. A subsequent five-seed live V1 experiment reserves scheduler-invisible tail pages and prepends them only to attention block tables. Native and full context recover 5/5 answers, disabled memory recovers 0/5, cleanup leaks are 0/5, and wrong memory controls follow the wrong fact in 5/5 cases. This closes controlled V1 generation on Metal. A second five-seed cohort combines native PRA with APC over concurrency 1–8: selected memory recovers 75/75 answers, alternating selected and ordinary requests score 40/40 on their expected outcome, wrong memory redirects 30/40 outputs, and throughput reaches 34.41 requests/s at concurrency eight. A selection-derived APC salt prevents the same visible query from reusing state created under different hidden memory; after the first selected request, APC supplies 128–144 prompt tokens. An official detached CUDA E2 path remains open. A CUDA E0 reproduction on an RTX 5060 Laptop GPU adds a 360-request natural-QA cohort and a 1,240-request warm load sweep. Selected text reaches 92.7 requests/s at concurrency 16; the independent full-context workload saturates at 12.0 requests/s and 1.15 s TTFT p99. A three-point pressure curve extends full context to 1,274 tokens: at concurrency 16 independent full context reaches 10.0 requests/s and 1.41 s TTFT p99, while 66-token selection remains above 91.9 requests/s and below 47.2 ms. A final matched-selection study uses the same routed QASPER, HotpotQA, and 2WikiMultiHopQA evidence for visible E0 and native E2 execution. E2 removes 90.6–93.0% of visible prompt tokens, and all 840 paired outputs are exact. A geometry audit explains an earlier 593/840 result: it compared full-prefix E0 prefill with query-only E2 prefill, so the two conditions traversed different fused-kernel shapes. Priming E0's source through APC and capturing E2 pages from vLLM's own source prefill holds that geometry fixed. Cold, warm, multi-query, and concurrency cost ratios are 0.985, 1.004, 0.993, and 0.981, respectively, where lower is better. Exact matched-quality promotion is therefore closed for this model and backend; connectors and CUDA remain open. Replications with Qwen3-1.7B, Llama-3.2-1B-Instruct, and Gemma-3-1B-IT also obtain 840/840 exact pairs each. Across all four checkpoints and 3,360 pairs, every completion is exact and all four execution-cost ratios remain within 3.1% of parity. An expanded Qwen3-1.7B confirmation uses 149 unique examples and preserves 2,086/2,086 pairs; its four macro cost ratios remain within 0.991–1.009. An expanded live storage study then obtains 80/80 lossless WARM output and array parity with restart recovery; int8 COLD is exact in only 17/80 sequences and is not promoted as a default. Finally, a vLLM 0.28 CUDA connector candidate stores source K/V by semantic identity and loads it into request-owned pages without recomputing the source content. A five-seed causal control obtains 5/5 exact E0/native pairs, 5/5 wrong-memory redirections, and zero disabled or cleanup leaks. With source-only geometry matched through APC, a 60-example natural cohort obtains 60/60 exact pairs and identical F1. Median candidate/E0 completion ratios are 0.964 on QASPER, 1.005 on HotpotQA, and 1.073 on 2Wiki. Because vLLM still reserves source-length placeholder slots, this closes native transfer and APC coexistence on CUDA, not scheduler-invisible detached E2 or its memory economics. A follow-up 45-request CUDA batch sweep covers

concurrency 1–8: every expected selected resource is recovered, no ordinary request receives forbidden selected detail, and shared-native throughput reaches 48.9 requests/s at concurrency eight.

1 Qualification Snapshot

The engine-specific question is whether query-addressed native pages add value beyond vLLM's strong selected-text E0 path and automatic prefix caching. The validated Metal path is E2: it consumes selected non-prefix K/V through native paged attention while keeping those pages scheduler-disjoint. Across 149 questions and four execution schedules, it preserves 2,086/2,086 E0 output pairs while removing 90.5–93.0% of visible prompt tokens. Its macro E2/E0 cost ratios are 1.009 cold, 0.997 warm, 1.003 multi-query, and 0.991 concurrent. This is near economic parity, not a general speedup. CUDA has a validated E0 path and a research-only native-transfer candidate. The candidate consumes semantic-keyed source K/V and preserves 60/60 matched outputs, but the scheduler still accounts for source-length placeholder slots. Detached page attachment, LMCache integration, and production CUDA E2 therefore remain open.

Integration level	E2 validated on vLLM-Metal; CUDA E0 plus prefix-shaped native-transfer candidate
Model/hardware	Four pinned checkpoints on Apple M5; TinyLlama-1.1B and CUDA E0 pressure on RTX 5060 Laptop GPU
Workload	QASPER, HotpotQA, and 2WikiMultiHopQA; frozen selections
Quality	Metal: 2,086/2,086 exact E0/E2 pairs; CUDA candidate: 60/60 exact, unchanged F1
Context reduction	Metal: 90.5–93.0% fewer visible tokens; CUDA candidate hides source content but retains placeholder slots
TTFT/ITL tails	`NOT_MEASURED` in the common matched cohort
Throughput	Metal E2/E0 inverse-cost ratio 0.991; CUDA candidate shared-native reaches 48.9 requests/s at $C = 8$
Peak memory	CUDA candidate 5.59 GiB allocated at $C = 8$; detached-page savings remain `NOT_MEASURED`
Reuse	APC-matched E0 and native HOT pages; LMCache `NOT_MEASURED`
Main limitation	Detached CUDA E2, LMCache, HBM/transfer telemetry, and online continuous batching

PRA primer. A PRA semantic resource has stable identity, version, authorization scope, and selectable token intervals. E0 freezes retrieval and renders the selected evidence as ordinary visible text. E1 transports logical resource metadata but still uses an ordinary model path. E2 consumes the same frozen selection as detached native K/V with correct positions, masks, cache topology, one normalization, and request-local cleanup. E3 additionally gives the engine scheduler ownership of placement, promotion, eviction, sharing, affinity, and batching. Prefix K/V and PRA K/V remain separate state classes.

Deployment recommendation. Best validated deployment: selected-text E0 on CUDA for general serving; pinned E2 on Metal when native non-disclosure or reuse is required. **Use when:** selection is stable and selected resources repeat across queries. **Do not use when:** a cold one-shot request gains no reuse or a deployment assumes generic vLLM labels imply native PRA. **Main remaining gate:** official CUDA page/connector integration under scheduler pressure.

The CUDA connector candidate is useful for integration research and semantic non-disclosure, but not yet a product default: its source K/V is native while its placeholder slots remain scheduler-visible, and its measured completion time does not consistently beat APC-backed E0.

2 Introduction

Long-running LLM services face two related but distinct memory problems. Sequential request KV must be allocated, paged, reused, and evicted. Separately, a retained corpus or session history need not participate in every query. The first problem is physical. The second is logical.

PagedAttention and vLLM address the physical problem by decoupling logical sequence positions from non-contiguous cache blocks and scheduling requests over the resulting pool [1]. PRA addresses the logical problem by separating address state from detail state and activating only selected non-prefix memory. The combined mapping is

$$\text{logical PRA memory} \rightarrow \text{query-selected logical blocks} \rightarrow \text{physical engine blocks}.$$

The middle arrow cannot be inferred from ordinary prefix equality. Two requests may share the same resource but select different intervals; two physical cache slots may hold the same immutable detail while using request-local position metadata.

This paper asks whether exposing that middle mapping to vLLM improves the quality-adjusted memory, latency, and throughput frontier beyond an optimized route-then-materialize baseline. The current iteration makes three contributions:

1. a stable block identity and validated residency lifecycle independent of vLLM's physical block numbers;
2. a vLLM adapter whose advertised PRA level is derived from an installed native executor, with tenant-scoped salt support at E0 and hidden-memory cache identity at E2;
3. native Metal paged-K/V kernel and live V1 generation studies, alongside the selected-text serving baseline, with parity, causality, cleanup, latency, and physical-sharing controls.

3 Two Independent Reuse Systems

Automatic Prefix Caching (APC) reuses an exact token prefix. PRA selects a non-prefix logical resource or interval. These mechanisms can coexist:

$$\underbrace{\text{stable conversation prefix}}_{\text{APC}} + \underbrace{\text{query-selected retained memory}}_{\text{PRA}}.$$

The security scopes are also independent. Prefix cache presence does not grant resource authorization, and a PRA block selected for one tenant cannot be reused by another merely because its physical bytes remain resident.

There is a less obvious correctness requirement. vLLM's APC key ordinarily depends on visible prompt tokens, whereas native PRA memory is absent from that prompt. Two requests with the same query but different selected memories would therefore collide. The bridge derives an opaque cache salt from the ordered logical selection, selected-token count, position base, consumer layers, and an optional deployment secret. Equal selection and geometry preserve warm reuse; any change creates a distinct namespace. The digest exposes neither resource names nor the secret. This protects correctness at the cost of losing APC sharing across different hidden-memory selections; reuse within one stable selection remains available.

Our E0/G10 baseline performs selection before the engine, injects the selected text into the current turn, and lets ordinary vLLM execute the request. This is the baseline a deeper integration must beat. Paged KV being active inside vLLM does not by itself make this path native PRA.

4 Logical PRA Block Contract

The implementation in `src/pra_hf/engine_memory.py` defines identity with the following fields:

Field group	Meaning
scope	tenant, optional session, and authorization scope
resource	resource identifier, immutable version, and record type
interval	half-open source-token interval $[i, j)$
model	immutable model revision, consumer layer, dtype, and K/V layout
policy	materialization profile and source-position/rebinding policy

The canonical JSON representation is hashed to produce an engine-safe logical key. Physical vLLM block numbers are deliberately absent. A resource update invalidates every matching physical realization without changing the identity of unrelated versions.

The state machine is

$$\textsc{IndexedOnly} \to \textsc{OffDevice} \to \textsc{Prefetching} \to \textsc{Resident},$$
$$\textsc{Resident} \leftrightarrow \textsc{Pinned}, \qquad \textsc{Resident} \to \textsc{Evictable} \to \textsc{OffDevice},$$

with an explicit terminal Invalid state. A transition to resident or pinned is rejected unless the engine bridge supplies at least one physical handle. Selection checks tenant and authorization scope before incrementing reuse counters. Snapshots report address bytes, logical detail bytes, resident detail bytes, indexed-only detail bytes, selections, and reuses separately.

5 Capability-Honest vLLM Adapter

The adapter in `src/pra_vllm/adapter.py` has two forms. Without a native executor it is an E0 OpenAI-compatible facade. It advertises APC and text fallback, but *not* logical references or native K/V. A deployment secret and tenant identifier derive a SHA-256 `cache_salt`; neither value is placed directly in the request.

With a native executor, the adapter becomes E2 and delegates generation, streaming, session close, and block-store access to that executor. This dependency-injection boundary is intentional. Metadata registration alone cannot promote the capability matrix. An executor must map selected identities to physical blocks and preserve K/V layout, GQA/MQA, masks, positions, consumer layers, request lifetime, and one correct attention normalization.

The concrete bridge is implemented in the vLLM-Metal native module. It maps each immutable logical key to ordinary native paged-cache pages and packs post-position K/V in $[\text{block}, \text{token}, H_{kv}, D]$ layout, builds one block table per request, and invokes the native Metal `paged_attention_primitive`. Multiple requests may reference one physical handle. This is the verified attention-kernel foundation used by the live V1 bridge below.

The next integration layer is implemented in `src/pra_vllm/v1_metadata.py`. A request registers immutable logical keys, selected block tables per cache group, selected-token count, source-position base, and consumer layers. At each V1 step the registry combines that object with scheduler-owned local cache starts and query-token counts without rewriting either. Thus two quantities remain explicit:

$$p_{q,0} = p_{\text{source}} + T_{\text{local}}, \qquad T_{\text{attn}} = T_{\text{selected}} + T_{\text{local}} + T_q.$$

Appending selected pages to ordinary scheduler block tables would instead make V1 count them as sequential prefix state and assign the wrong query positions. The registry validates immutable request registration and explicit cleanup. `src/pra_vllm/v1_native.py` supplies the corresponding model-runner hook: it extends the physical Metal cache with a reserved tail, writes immutable selected pages there, and prepends those page IDs only to request-local attention metadata. Scheduler slot mappings, sequential token counts, and ordinary prefix identities remain unchanged. Query positions advance by the source-position base. This first E2 path requires page-aligned selections, one cache group, all attention layers, and non-TurboQuant K/V.

5.1 CUDA connector candidate

The CUDA path uses vLLM's V1 KV-connector seam rather than the Metal model-runner hook. `src/pra_vllm/cuda_protocol.py` encodes a request-scoped `store` or `load` command containing a URL-safe logical resource key and block-aligned source-token count. `src/pra_vllm/cuda_connector.py` keeps that logical identity outside token hashing. During a store request it extracts each attention layer's source slots from vLLM's paged CUDA cache and persists the contiguous K/V tensor plus a completion manifest. During a load, the scheduler allocates isolated request pages, the connector copies the selected tensors into those pages before attention, and vLLM evaluates only the query suffix. Ordinary request cleanup releases the destination pages; the semantic object remains separately addressable.

This path preserves native attention, K/V positions, and one ordinary self-attention normalization. It also supports causal substitution: the same query can load a different logical resource without changing source-token content, because placeholder IDs occupy those positions but are never evaluated. It is nevertheless weaker than detached PRA. vLLM's connector contract loads external K/V as a computed prompt prefix, so the scheduler allocates and counts one placeholder slot per source token. Selection-specific salts prevent accidental APC reuse for native-load requests, while the E0 reference primes the same source-only geometry in its ordinary APC namespace. We therefore label this path *E2 candidate, prefix-shaped*, rather than promoting generic CUDA to detached E2.

6 Environment and Method

The remote host was an Apple M5 MacBook Pro with 16 GB unified memory, macOS 26.4.1, and Metal 4. The environment used Python 3.12.7, vLLM 0.28.0+cpu, and vLLM-Metal 0.3.0.dev20260828085745 at revision `14705ad974863f68d00315655514f200366441bf`. Although the package version contains `+cpu`, startup logs show the Metal plugin, MLX model loader, and native M5 paged-attention kernels. We therefore report both package and backend provenance.

The model was `mlx-community/Qwen3-0.6B-4bit` at revision `73e3e38d981303bc594367cd910ea6eb48349da8`. The server used a 2,048-token maximum context, one sequence, 20% memory reservation, and explicit APC. It allocated 1,589.1 MB for 866 KV blocks of 16 tokens, or 13,856 tokens.

Table 1: vLLM-Metal smoke results. TTFT is milliseconds. "Warm cached" is not available per request in the SSE usage payload.

Condition	Exact	Prompt	Cold TTFT	Warm TTFT	Warm cached
None	0%	24	875.9	42.2	n/a
Prefix	0%	333	116.6	49.4	n/a
Selected	100%	56	62.7	42.9	n/a
Prefix + selected	100%	365	80.7	46.2	n/a
Full context	100%	1577	776.6	52.9	n/a

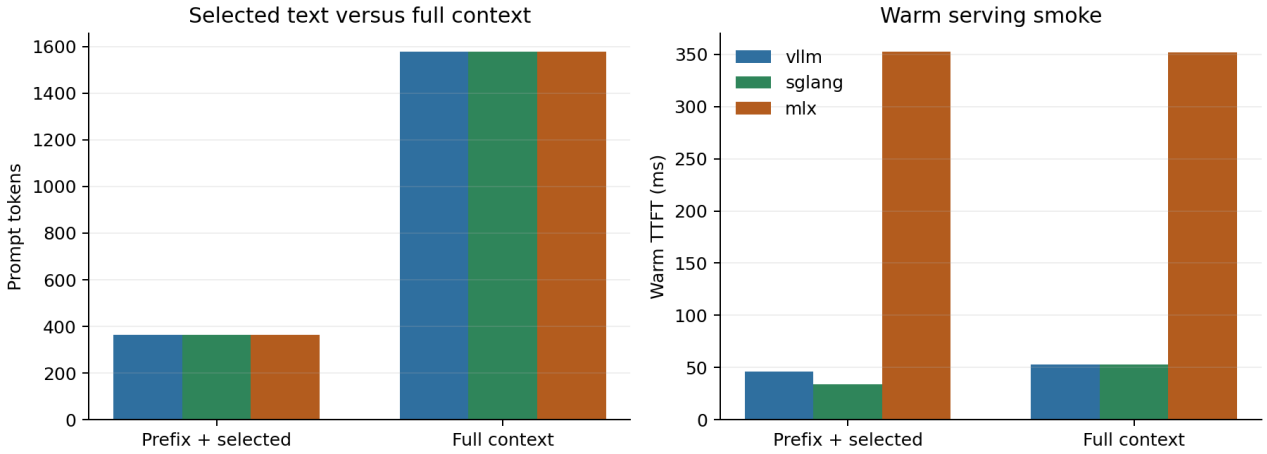

Figure 1: Shared prompt-size and warm-TTFT calibration. Timings across engines are not direct speed rankings: SGLang used an unquantized MLX conversion, whereas vLLM and MLX-LM used the 4-bit checkpoint.

The fixed synthetic task contains one authoritative codeword and twelve distractor records. Five conditions separate no memory, a stable prefix, selected evidence, stable prefix plus selected evidence, and full context. Each condition is repeated five times contiguously. The first request is reported as cold within the running server; the remaining four form the warm mean. Streaming Server-Sent Events determine TTFT. Twenty-five requests are enough for compatibility and mechanism smoke evidence, not tail percentiles or serving claims.

The CUDA connector experiments run vLLM 0.28.0 with FlashAttention 2 and TinyLlama-1.1B-Chat-v1.0 in BF16 on an NVIDIA RTX 5060 Laptop GPU with 8 GB VRAM under WSL. Eager V1 execution uses 16-token cache blocks and disables the hybrid cache manager. The controlled cohort has five seeds. The natural cohort uses the frozen portable manifest's 20 QASPER, 20 HotpotQA, and 20 2WikiMultiHopQA examples. Selected sources are capped at 384 tokens and padded to whole blocks. E0 and native conditions share source-only encoding geometry; E0 recovers it through APC, while the connector reloads it by semantic key. Greedy decoding is capped at 24 tokens. Offline completion time, exact output pairing, EM, F1, APC tokens, and persisted K/V bytes are measured; TTFT, ITL, PCIe traffic, and K/V-only HBM are NOT MEASURED.

7 Results

Table 1 first validates the control. Neither no-memory condition recovers the codeword, whereas selected and full context recover it in every request. Prefix plus selected memory uses 365 prompt tokens versus 1,577 for full context, a reduction of

$$1 - \frac{365}{1577} = 76.9\%.$$

Cold TTFT is 80.7 ms for prefix plus selected memory and 776.6 ms for full context. This large difference includes first-touch effects and should not be treated as a steady-state speedup. Warm TTFT narrows to 46.2 versus 52.9 ms. The engine log reports aggregate APC hit rates of 75.3% and 83.4% during the run, demonstrating that selected context did not disable prefix caching.

Table 2: Native vLLM-Metal paged attention. Each row aggregates five seeds and four concurrency levels. Cold includes first invocation for that row; warm is an immediate repeat. Sharing savings are at concurrency eight.

Selected tok.	Fraction	Cold ms	Warm ms	Max error	Shared MB saved
16	0.20%	2.769	0.489	0.003906	0.44
64	0.78%	0.602	0.523	0.0009766	1.75
256	3.12%	0.675	0.548	0.0009766	7.00
512	6.25%	0.722	0.642	0.0007324	14.00

Table 3: Five-seed live vLLM-Metal V1 generation controls. "Control success" means recovery for positive conditions, failure for disabled memory, absence of post-request leakage, or following the deliberately wrong memory.

Condition	Control success	Mean latency ms
Full visible context	100%	197.3
Selected native K/V	100%	159.7
Disabled memory (must fail)	100%	–
Post-cleanup isolation	100%	–
Wrong-memory causality	100%	–

Figure 1 shows the stable prompt-size reduction across all three tokenizers and the engine-specific latency scale. The comparable result is within-engine selected versus full context, not the absolute height of different-engine bars.

7.1 Native paged selected K/V

Table 2 tests the physical mechanism directly. Selected K/V occupies native vLLM-Metal pages and is consumed by the same paged-attention primitive used for sequential cache blocks. The output agrees with a dense MLX reference at every tested geometry. Warm latency changes smoothly with selected set size after compilation, and one immutable allocation is shared by all requests. These are positive block-execution and sharing results; the next experiment tests their use during generation.

7.2 Live V1 generation

Table 3 carries 32 selected native tokens through actual V1 generation while only 15 query tokens are visible. Native selected K/V and full visible context both recover the target in all five seeds. Query-only generation never recovers it, the request after native cleanup shows no leak, and substituting a wrong native memory changes the answer to that memory's code in every seed. Mean completion time is 159.7 ms for native memory and 197.3 ms for full context; this small controlled cohort is a mechanism timing, not a throughput claim. Active selected K/V is 3.50 MiB. The experiment closes generation, causal influence, and request cleanup on this pinned Metal path.

7.3 Native PRA, APC, and concurrent V1 requests

Table 4 closes the previously separate APC and native-generation paths. vLLM reports 128–144 ordinary cached prompt tokens, and the bridge observes the same scheduler-owned boundaries before adding 32 selected tokens at the attention boundary. Thus selected pages are neither counted as APC hits nor inserted into the ordinary prefix namespace. Native recovery is 75/75 as concurrency increases from one to eight. In the strongest isolation condition, alternating rows in the same eight-request batch receive selected pages: all 20 selected rows recover the code and all 20 adjacent ordinary rows do not. A subsequent disabled batch leaks in 0/40 requests.

Figure 2 shows mean throughput increasing from 4.76 requests/s at concurrency one to 34.41 at concurrency eight. The first selected request is cold in its selection-specific APC namespace; subsequent concurrency levels reuse 128–144 visible-prefix tokens. This is offline V1 batching on one Apple M5, not an HTTP arrival-process or tail-latency benchmark. Wrong memory redirects 30/40 outputs to its counterfactual code; the imperfect rate limits the strength of that prompt-level causal control, while the earlier short-prefix experiment supplies the cleaner 5/5 result.

M5 NAX replication. We independently restored vLLM 0.28.0 and the pinned vLLM-Metal source on a 16 GB Mac17,2, loaded the prebuilt M5 NAX and paged-attention libraries, and reran the same five-seed schedule. Native

Table 4: Five-seed APC and concurrency cohort. Success denotes exact recovery for selected requests, non-recovery for ordinary/disabled requests, and exact redirection for wrong-memory controls. Throughput is the mean across seeds.

Condition	n	Success	Requests/s	APC tokens
Native PRA + APC, $C = 1$	5	5/5	4.76	0–0
Native PRA + APC, $C = 2$	10	10/10	10.33	128–144
Native PRA + APC, $C = 4$	20	20/20	19.99	128–144
Native PRA + APC, $C = 8$	40	40/40	34.41	128–144
Mixed selected / ordinary	40	40/40	35.12	144–144
Post-native disabled isolation	40	40/40	34.31	144–144
Wrong-memory redirection	40	30/40	34.51	144–144

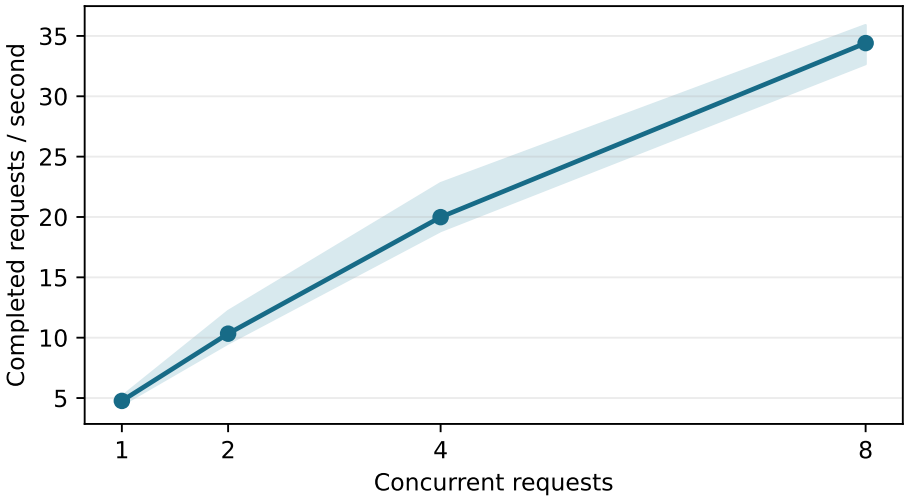

Figure 2: Live native-PRA V1 throughput with APC. The line is the five-seed mean; the band spans the observed seed range. Recovery is 100% at every concurrency shown.

recovery again reaches 75/75; mixed selected/ordinary isolation is 40/40 and post-request isolation is 40/40. Mean throughput is 4.79, 9.90, 18.75, and 30.71 requests/s at concurrency 1, 2, 4, and 8. This is a successful correctness replication but not a throughput improvement over the earlier 34.41 requests/s run. The offline V1 `RequestOutput` objects expose APC counts but no per-request arrival or first-token timestamps on this plugin revision, so TTFT, queue, and TPOT remain explicitly unavailable rather than inferred from batch completion. The machine-readable receipt records the engine and source revisions, Apple chip identity, physical memory, every output, and nullable latency fields.

7.4 CUDA E0 reproduction and saturation boundary

We separately run vLLM 0.28.0 with TinyLlama-1.1B-Chat on an NVIDIA RTX 5060 Laptop GPU. The installed V2 runner requires unified virtual addressing, which this WSL configuration does not expose; setting `VLLM_USE_V2_MODEL_RUNNER=0` selects the supported V1 runner. This is an E0 selected-text reproduction, not the Metal native-page E2 path above. The server enables APC, reserves 70% of device memory, and reports a 149,424 token KV-cache capacity.

Selected context improves token F1 over no context on all three datasets while using 402–437 visible prompt tokens. Full context uses 939–1,568 tokens and is better on HotpotQA and QASPER, exposing the selector's remaining evidence gap. TinyLlama's absolute QA quality is low, so this cohort is most useful as a natural-input transport and latency control. Selected median TTFT is 23.5–28.7 ms, compared with 37.2–63.3 ms for full context.

Table 5: CUDA natural-QA E0 cohort: 20 examples per dataset and two repeats. Times are milliseconds. Retrieval is frozen before the three execution conditions.

Dataset	Context	F1	Contain.	Prompt tok.	TTFT p50	TTFT p95
2Wiki	None	0.066	0.150	38	16.3	19.4
2Wiki	Selected	0.105	0.250	402	23.5	41.1
2Wiki	Full	0.095	0.300	939	37.2	100.2
hotpotqa	None	0.069	0.000	44	16.4	28.6
hotpotqa	Selected	0.089	0.200	437	28.7	43.3
hotpotqa	Full	0.132	0.350	1440	51.2	117.1
qasper	None	0.041	0.000	32	16.3	24.7
qasper	Selected	0.069	0.050	419	27.8	41.4
qasper	Full	0.087	0.050	1568	63.3	119.2

Table 6: Warm CUDA E0 load endpoints. Each row has ten waves; selected and full context use the same answer-bearing record.

Workload	Context	C	Succ.	Req/s	Out tok/s	TTFT p50	TTFT p99	ITL p99
Shared	Selected	1	1.00	7.13	142.6	15.8	17.9	6.6
Shared	Selected	8	1.00	50.13	1002.6	26.8	31.0	6.9
Shared	Selected	16	1.00	92.66	1853.2	36.1	42.1	6.9
Independent	Selected	1	1.00	7.14	142.8	16.2	18.0	6.5
Independent	Selected	8	1.00	50.14	1002.8	26.9	30.1	6.8
Independent	Selected	16	1.00	93.53	1870.6	34.7	43.0	6.9
Shared	Full	1	1.00	7.06	141.2	17.9	22.2	6.5
Shared	Full	8	1.00	43.51	870.3	39.1	46.3	7.8
Shared	Full	16	1.00	72.81	1456.3	53.9	68.3	8.8
Independent	Full	1	1.00	6.96	139.2	17.8	21.4	6.8
Independent	Full	8	1.00	42.11	842.3	39.7	45.8	8.0
Independent	Full	16	1.00	11.98	239.6	682.4	1149.0	64.4

One no-context QASPER request incurred a 19.3 s first-token outlier; the warm load experiment below separates this cold disturbance from steady load.

At concurrency 16, selected text reaches 92.7 and 93.5 requests/s for shared and independent resources, respectively, with 42–43 ms TTFT p99. Shared full context reaches 72.8 requests/s and 68.3 ms p99. Independent full context instead falls from 42.1 requests/s at concurrency eight to 12.0 at 16, while TTFT p99 rises from 45.8 ms to 1.15 s and ITL p99 from 8.0 to 64.4 ms. The distinction is important: repeated prefixes remain manageable under APC, but many independently primed long prefixes exhaust the tested scheduling/cache frontier. Selected E0 avoids that collapse in this bounded campaign. It does not establish a native-E2 speedup, which still requires CUDA page attachment.

7.5 CUDA context-pressure curve

We next hold the selected prompt at 66 tokens and increase the full visible prompt from 646 to 1,038 and 1,274 tokens. Every condition uses concurrency 16, 20 measured waves after explicit warmup, and the same answer-bearing record.

Selected throughput remains 91.9–93.5 requests/s and TTFT p99 remains 42–47 ms across the three points. Shared full context declines modestly from 79.1 to 72.1 requests/s, with TTFT p99 growing from 58 to 68 ms because APC reuses one identity. Independent full context crosses a much sharper boundary: throughput falls from 18.5 to 10.0 requests/s and TTFT p99 rises from 691 ms to 1.41 s. All outputs remain exact, so this curve isolates scheduling and cache pressure rather than a quality/termination artifact. It supports bounded selected-text serving on this CUDA configuration; it still does not measure native non-prefix K/V attachment.

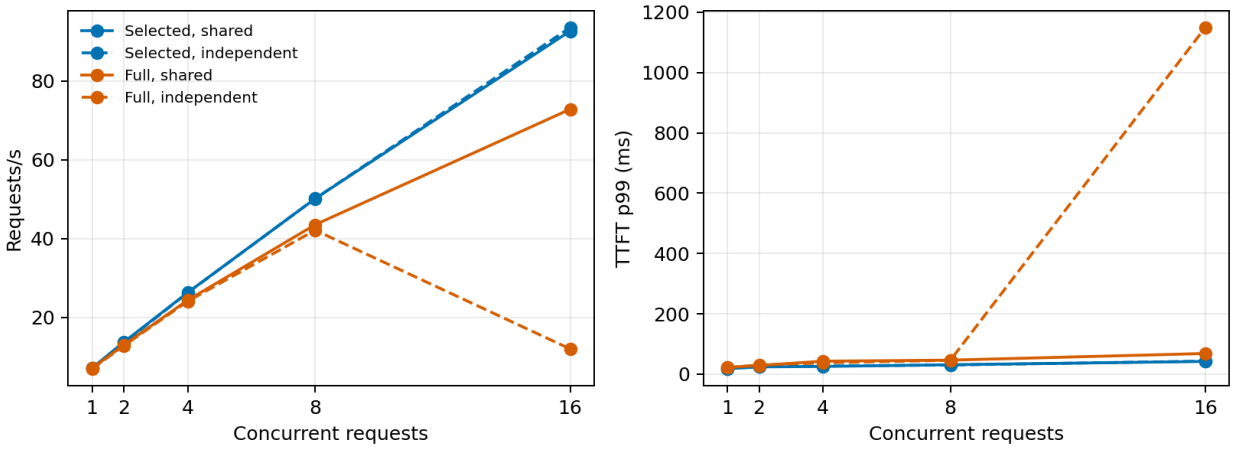

Figure 3: CUDA E0 throughput and TTFT tail under shared and independent resources. The independent full-context curve crosses the tested saturation boundary at concurrency 16.

Table 7: CUDA context pressure at concurrency 16. All controlled outputs are exact; the independent-resource rows vary prefix identity across requests.

Size	Workload	Context	Prompt tok.	Succ.	Req/s	Out tok/s	TTFT p99
Small	Shared	Selected	66	1.00	92.87	1857.4	44.7
Small	Independent	Selected	66	1.00	92.99	1859.8	42.8
Small	Shared	Full	646	1.00	79.11	1582.2	58.2
Small	Independent	Full	646	1.00	18.54	370.9	691.4
Medium	Shared	Selected	66	1.00	92.66	1853.2	42.1
Medium	Independent	Selected	66	1.00	93.53	1870.6	43.0
Medium	Shared	Full	1038	1.00	72.81	1456.3	68.3
Medium	Independent	Full	1038	1.00	11.98	239.6	1149.0
Large	Shared	Selected	66	1.00	92.40	1847.9	46.4
Large	Independent	Selected	66	1.00	91.93	1838.5	47.2
Large	Shared	Full	1274	1.00	72.08	1441.7	68.0
Large	Independent	Full	1274	1.00	10.00	200.1	1407.7

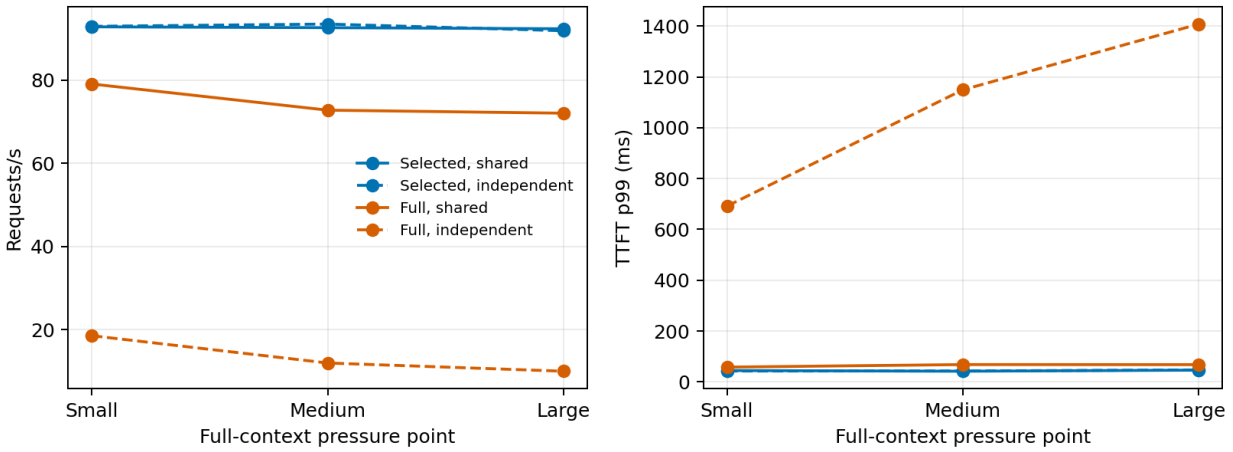

Figure 4: CUDA peak-load throughput and TTFT tail as full-context pressure increases. Selection stays fixed at 66 prompt tokens.

Table 8: Cross-engine matched-selection quality. All parity pools 840 paired requests per engine across four schedules. Paper 6 uses the vLLM-Metal rows; the other rows provide the common reference.

Engine	Dataset	Visible E0/E2	Reduction	Cold F1 E0/E2	Cold parity	All parity
mlx-lm	2wikimultihopqa	375/33	91.1%	0.067/0.067	100.0%	100.0%
mlx-lm	hotpotqa	415/39	90.6%	0.087/0.087	100.0%	100.0%
mlx-lm	qasper	399/28	93.0%	0.110/0.110	100.0%	100.0%
sglang-mlx	2wikimultihopqa	375/33	91.1%	0.074/0.074	100.0%	100.0%
sglang-mlx	hotpotqa	415/39	90.6%	0.084/0.084	100.0%	100.0%
sglang-mlx	qasper	399/28	93.0%	0.105/0.105	100.0%	100.0%
vllm-metal	2wikimultihopqa	378/33	91.2%	0.079/0.079	100.0%	100.0%
vllm-metal	hotpotqa	417/39	90.6%	0.074/0.074	100.0%	100.0%
vllm-metal	qasper	400/28	93.0%	0.101/0.101	100.0%	100.0%

Table 9: Macro E2/E0 execution-cost ratios; lower is better. Cold includes one-time ingestion, warm and multi-query use median completion, and concurrent uses inverse requests/s. The final column is E2 time to usable context.

Engine	Cold	Warm	Multi-query	Concurrent	E2 usable ms
mlx-lm	1.130	1.155	1.148	1.132	51.2
sglang-mlx	0.976	1.136	1.140	1.122	55.0
vllm-metal	0.985	1.004	0.993	0.981	86.0

7.6 Matched selected text versus native K/V

The final experiment freezes 20 routed examples from each of QASPER, HotpotQA, and 2WikiMultiHopQA across five seeds. Within an example, E0 and E2 receive the same source-token sequence, question, greedy decoder, and 24-token output budget. vLLM first encodes the selected source alone for both conditions. E0 retains those ordinary pages through APC and exposes the source as prompt text; E2 captures the corresponding vLLM-native source pages and attaches an immutable copy outside APC. Both conditions consequently prefill only the question suffix during the measured request. Each condition follows four schedules: cold one-shot, two warm repeats, three query formulations over the same resource, and an eight-request continuous-batching wave. The selector runs only once, so any output difference occurs after selection.

E2 reduces mean visible tokens by 90.6–93.0%. Across all schedules, exact output parity is 100% on 2Wiki, HotpotQA, and QASPER: 840/840 paired requests, with zero signed or absolute pairwise F1 change. MLX-LM and SGLang-MLX obtain the same 840/840 result. This is transport equivalence at a frozen selector, not evidence that the model answers every question correctly.

The macro cold end-to-end cost ratio is 0.985 after 86.0 ms mean E2 time to usable context. Warm is 1.004, multi-query is 0.993, and the concurrency-eight inverse-throughput ratio is 0.981. Thus vLLM-Metal is economically near parity in this small offline workload and slightly favors E2 in three of four macro regimes. The differences are too small for a broad speed claim. The offline V1 API emits the 24-token completion as one engine result, so TTFT and ITL are reported as unavailable rather than inferred from completion time.

We repeat the complete 840-pair protocol with Qwen3-1.7B, Llama-3.2-1B-Instruct, and Gemma-3-1B-IT. The selector, source cap, schedules, 4-bit format, and Metal backend remain fixed while model architecture and tokenizer family vary. Table 10 reports the result.

All three replications preserve all 840 paired outputs and zero pairwise F1 change. Llama's macro ratios are 0.992 cold, 0.991 warm, 1.001 multi-query, and 0.969 concurrent; Gemma's are 0.998, 1.007, 0.988, and 0.991. Thus the corrected geometry and near-parity economics are not confined to Qwen or one tokenizer. The experiment still shares the 4-bit format, Apple host, and Metal backend, so CUDA and other serving implementations remain external-validity boundaries.

We finally expand Qwen3-1.7B to 50 sampled examples per dataset (49 unique for QASPER). Across 149 unique questions and the same four schedules, E0 and E2 match in 2,086/2,086 paired generations. The Wilson 95% lower bound is 0.9982 over all pairs and 0.9749 for the 149 cold pairs alone. Visible-token reductions are 90.5–93.0%, while macro cold, warm, multi-query, and concurrent cost ratios are 1.009, 0.997, 1.003, and 0.991. Thus increasing the natural cohort does not reveal a semantic or economic regression on this Metal backend; it also does not substitute

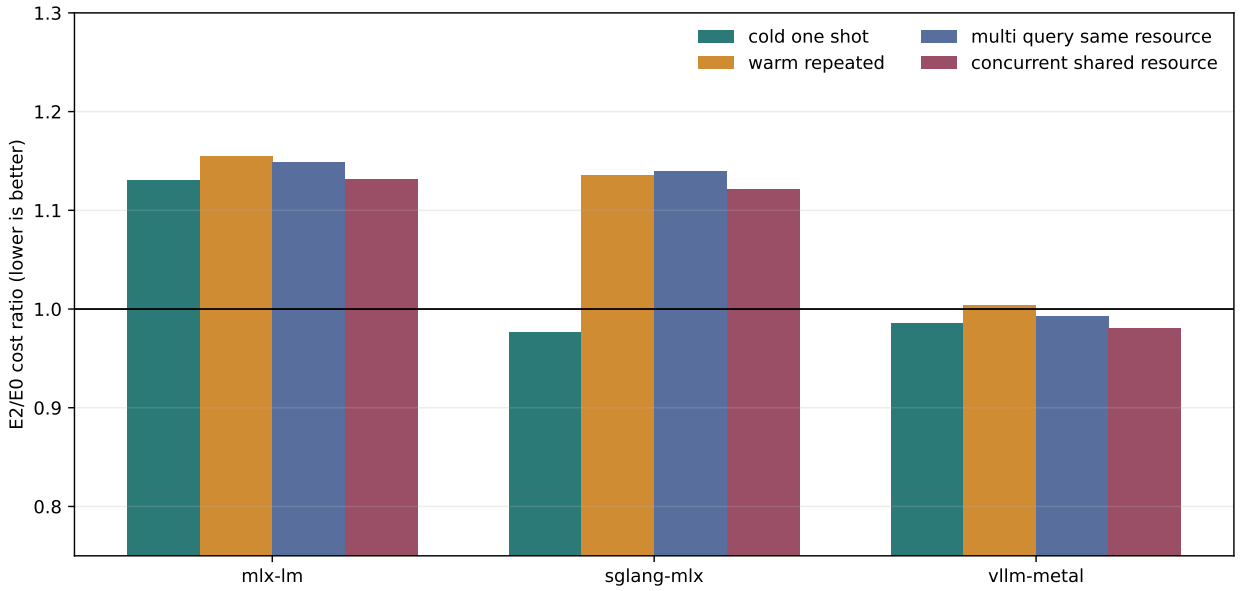

Figure 5: Macro representation cost after freezing retrieval. Lower is better; concurrent cost is inverse throughput. vLLM-Metal is near E0 economic parity across all four schedules.

Table 10: Matched E2/E0 execution across four checkpoints and three model families. Lower cost ratios are better. Each row contains 840 paired requests.

Model	Exact pairs	Cold	Warm	Multi-query	Concurrent
Qwen3-0.6B-4bit	840/840	0.985	1.004	0.993	0.981
Qwen3-1.7B-4bit	840/840	1.007	0.992	0.994	0.987
Llama-3.2-1B-Instruct-4bit	840/840	0.992	0.991	1.001	0.969
gemma-3-1b-it-4bit	840/840	0.998	1.007	0.988	0.991

for CUDA continuous-batching and HBM-transfer validation.

7.7 Shared semantic storage boundary

Paper 4.5 now supplies the engine-neutral HOT/WARM/COLD/SOURCE policy used above the vLLM bridge. In this mapping, only HOT is a native vLLM-Metal page set. WARM and COLD are scheduler-invisible PRA backends; promotion allocates PRA pages, restores the fingerprint-compatible selected K/V, and attaches their identity to request-local metadata. Demotion first unpins and releases those pages. Record type, task closure, persistence grace, and quota eviction do not belong in the V1 executor, and selected detail is never returned to APC as ordinary sequential K/V.

The shared five-seed-per-engine lifecycle control obtains exact lossless tier round-trip and SOURCE reconstruction in all 20 policy runs. It is not a second vLLM generation benchmark: the live V1 and matched E0/E2 experiments remain the physical and semantic evidence for this engine. LMCache or a connector can replace the WARM byte backend only after identity, request isolation, one-copy attachment, and fingerprint rejection pass unchanged.

We then connect the shared manager directly to `VLLMPageHotBridge`. Three QASPER selections traverse native HOT pages, a layer-segmented mmap WARM store, int8 COLD, and manager restart. Request IDs pin pages across complete generation and cleanup returns them only to the PRA reserve, never APC. Lossless WARM reproduces 3/3 HOT outputs and restart promotion succeeds. Median request latency is 356 ms HOT and 699 ms after a WARM hit (1.96×); the 362 ms median background demotion is excluded. COLD promotion plus generation is 712 ms, while background quantization and persistence costs 1,016 ms median.

Int8 COLD changes all 3/3 vLLM sequences, preserves the first token in 2/3, and has a four-token mean

Table 11: Shared three-example live storage probe. WARM is lossless; COLD uses explicit per-array int8 and is not the default product profile.

Engine	WARM exact	int8 exact	int8 first	Prefix	WARM/HOT	COLD/HOT
mlx-lm	100%	0%	67%	1.3	1.97	1.84
sglang-mlx	100%	0%	67%	0.7	1.90	1.89
vllm-metal	100%	0%	67%	4.0	1.96	2.00

Table 12: Expanded vLLM-Metal lifecycle cohorts. p50/p95 include promotion and generation; int8 is diagnostic.

Engine	Model	Data	n	WARM exact	int8 exact	HOT p50	WARM p50	WARM p95
vllm-metal	Llama-3.2-1B-Instruct-4bit	qasper	5	100.0%	60.0%	299	428	453
vllm-metal	Qwen3-0.6B-4bit	2wikimultihopqa	20	100.0%	30.0%	351	738	803
vllm-metal	Qwen3-0.6B-4bit	hotpotqa	20	100.0%	35.0%	366	737	832
vllm-metal	Qwen3-0.6B-4bit	qasper	20	100.0%	10.0%	360	714	779
vllm-metal	Qwen3-1.7B-4bit	qasper	20	100.0%	10.0%	383	769	824
vllm-metal	gemma-3-1b-it-4bit	qasper	5	100.0%	100.0%	457	585	623

common prefix. The mean answer-F1 change is -0.005 on this small, low-accuracy cohort, which is insufficient to certify quality. The vLLM product path therefore keeps COLD quantization opt-in even though its byte reduction is useful.

The expanded protocol covers 20 examples for each 0.6B dataset and 20 QASPER examples at 1.7B. Lossless WARM output and restored native arrays are both 80/80 exact, and all four manager instances recover after restart. Int8 COLD is exact in 17/80 sequences. Table 12 also shows that WARM request p50 remains 1.98–2.10 times HOT, so storage correctness is closed while synchronous promotion economics remain open.

7.8 Capture/replay and position audit

We then copy ordinary source-prefill pages from vLLM's own paged cache and replay them through the scheduler-invisible PRA tail. Every native request uses a selection-derived APC salt, preventing query-cache reuse across audit conditions. Table 13 separates the first decoding decision from exact 24-token sequence equality.

The nominal base is decisively better than zero and preserves the first token in 5/5 cases. The apparent residual gap was not a page-count or position-base defect. Full-prefix E0 and query-only E2 invoked different prefill shapes; even full-prefix E0 and an APC-restored E0 diverged in 2/5 sequences while preserving the first token in 5/5. Once E0 uses the same APC-restored query-only geometry, captured E2 pages match all 24 output tokens in 5/5 cases. The full 60-example, 840-pair rerun then reproduces exact parity in every regime. This audit is a warning for engine benchmarking: numerical drift from different kernel geometry must not be attributed to a hidden-memory representation.

7.9 Controlled residency pressure

The engine-neutral residency manager was also exercised on five deterministic 120-request traces with a 14 MB budget and a 36 MB working set. This is a control-plane simulation; it does not substitute for engine telemetry. Size-aware LRU reduces mean reload amplification from 0.643 to 0.397, a 38.3% relative reduction. Reuse-count and reload-cost policies reach 0.445 and 0.437. The result confirms that selected-memory lifecycle policy can matter even before learned eviction, while leaving its effect on real TTFT open.

7.10 CUDA semantic K/V connector

The controlled CUDA gate first uses 32 source tokens and five independently named resources. Full E0 and semantic-keyed native loading produce identical token sequences in 5/5 cases and recover the target in 5/5. Disabled and post-cleanup requests leak the target in 0/5. Replacing the logical key with a same-shaped wrong resource redirects the answer to that resource's code in 5/5. This closes native consumption, causal influence, and request isolation for the connector candidate.

The initial natural comparison preserved 56/60 exact sequences. Its four differences were punctuation, stopping, or continuation-boundary changes, not cross-request leakage. As in the earlier Metal audit, source-only native encoding and full source-plus-query E0 prefill used different FlashAttention shapes. We therefore prime the E0

Table 13: Five-example QASPER capture/replay diagnostic. Prefix is the mean number of generated tokens equal to E0 before the first divergence.

Native source and position	Exact	First token	Mean prefix
separate generic encoder, nominal base	2/5	5/5	15.6
captured vLLM pages, base zero	0/5	0/5	0.0
captured vLLM pages, nominal −1	1/5	4/5	9.0
captured vLLM pages, nominal base	3/5	5/5	17.8
captured vLLM pages, nominal +1	2/5	4/5	13.0
captured pages, APC-matched E0 geometry	5/5	5/5	24.0

Table 14: Fixed-budget residency pressure over five seeds. Reload amplification is physical loads divided by requests; lower is better.

Policy	Mean loads	Mean evictions	Reload amplification
lru	77.2	73.8	0.643
size_aware_lru	47.6	43.6	0.397
reuse_count	53.4	49.4	0.445
reload_cost	52.4	48.4	0.437

source through APC and rerun the frozen selection with identical source-only geometry. Table 15 reports the corrected result.

All 60 corrected natural pairs are token-exact, so EM and F1 are unchanged. The completion ratios are 0.964, 1.005, and 1.073: one small win, one tie, and one regression. Native source objects occupy 7.62–8.22 MiB for mean selections of 354–382 tokens. Median native ingestion is 55.5–58.2 ms, versus 32.3–32.6 ms for the APC source prime. These values include synchronous host-file serialization or loading and do not isolate PCIe transfer. The candidate therefore proves portable CUDA-native K/V substitution and APC coexistence, but not lower HBM, faster warm reuse, or scheduler-native E3. Because its placeholder prefix preserves source-length scheduler occupancy, it also does not yet realize PRA's detached-page scaling objective.

7.11 CUDA connector concurrency and isolation

We next batch the same semantic-keyed source objects at concurrency 1, 2, 4, and 8. Three conditions distinguish immutable sharing, alternating native and ordinary requests, and a wrong-memory control whose expected answer belongs to the intentionally substituted resource. Table 16 and Figure 7 summarize 45 requests.

All 38 requests expected to recover a selected resource do so, including the wrong-memory causal condition, and all seven adjacent ordinary requests avoid the selected code. Thus 45/45 requests have the expected memory scope and forbidden leakage is zero. Shared-native throughput rises from 2.8 requests/s at concurrency one to 48.9 at eight; mixed native/ordinary reaches 44.2 requests/s at eight. Peak allocated memory changes only from 5,591.7 to 5,598.2 MiB across the sweep because the engine's preallocated cache dominates these small request batches. This closes controlled concurrent isolation for the prefix-shaped CUDA candidate. It does not establish scheduler-invisible native pages, LMCache reuse, online p95/p99, or an HBM saving over E0.

Table 15: RTX 5060 CUDA connector candidate against geometry-matched APC E0. The native column loads semantic-keyed K/V into request-owned pages. Ratio is native/E0 median completion; lower is better.

Dataset	n	Exact pair	F1 E0/cand.	Completion E0/cand. (ms)	Ratio	Native MiB
QASPER	20	100%	0.131/0.131	297.3/286.5	0.964	8.11
HotpotQA	20	100%	0.208/0.208	275.5/276.8	1.005	8.22
2Wiki	20	100%	0.108/0.108	281.0/301.4	1.073	7.62

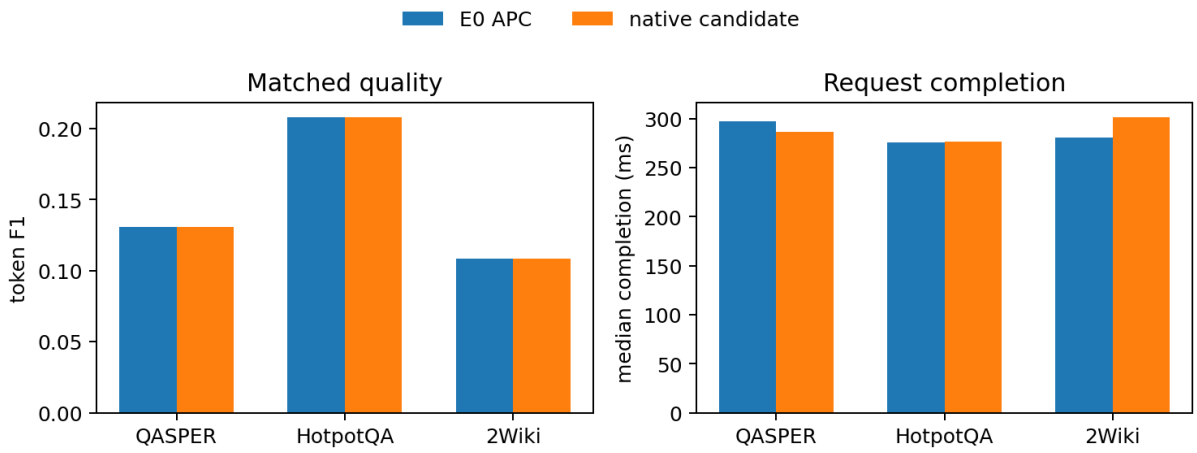

Figure 6: The CUDA candidate preserves matched F1 exactly. Completion time is near APC E0 but has no consistent advantage across the three datasets.

Table 16: Offline vLLM V1 CUDA connector batches. Recovery is relative to each condition's intended resource; leakage means that an ordinary request received selected native detail. Peak is PyTorch CUDA allocation for the engine.

Condition	C	Recovery	Leakage	requests/s	Peak MiB
shared native	1	1.000	0.000	2.8	5592
mixed native/ordinary	1	1.000	0.000	5.2	5592
wrong-memory control	1	1.000	0.000	6.7	5592
shared native	2	1.000	0.000	13.4	5593
mixed native/ordinary	2	1.000	0.000	14.0	5593
wrong-memory control	2	1.000	0.000	10.6	5593
shared native	4	1.000	0.000	22.4	5595
mixed native/ordinary	4	1.000	0.000	31.9	5595
wrong-memory control	4	1.000	0.000	25.5	5595
shared native	8	1.000	0.000	48.9	5598
mixed native/ordinary	8	1.000	0.000	44.2	5598
wrong-memory control	8	1.000	0.000	47.9	5598

8 Promotion Gates

Gate	Status	Evidence
environment	closed	vLLM V1 with native Metal paged attention runs reproducibly
black-box semantics	smoke closed	selected and full context agree on the fixed task
prefix coexistence	controlled closed	128–144 APC tokens after first selection-scoped request
native block attention	controlled closed	80 Metal kernel rows; dense-reference parity
physical sharing	controlled closed	one immutable handle at concurrency 1–8
V1 metadata contract	controlled closed	selected block tables and positions remain scheduler-disjoint
live V1 generation	controlled closed	positive, disabled, wrong-memory, and cleanup controls over five seeds
concurrent isolation	controlled closed	20/20 selected recoveries and 0/20 adjacent ordinary recoveries
native APC identity	controlled closed	selection-derived salt; 75/75 selected and 0/40 disabled recoveries
matched natural QA	controlled closed	3,360/3,360 exact outputs across Qwen, Llama, and Gemma; zero pairwise F1 change
shared storage lifecycle	controlled closed	live pages; WARM 80/80 exact; restart recovery; int8 COLD 17/80
CUDA connector candidate	research closed	5/5 causal controls and 60/60 natural pairs; prefix-shaped slots remain
CUDA connector concurrency	research closed	45/45 scoped outcomes through $C = 8$; zero forbidden leaks
engine-native serving	partial	Metal native APC plus CUDA native transfer; detached CUDA, LMCache, and online tails remain open
robustness	partial	four Metal checkpoints plus TinyLlama CUDA; two backend families

The Metal backend now accepts selected non-prefix K/V at its native paged-attention boundary and carries scheduler-invisible pages through V1 generation. Native pages now coexist with scheduler-owned APC state through

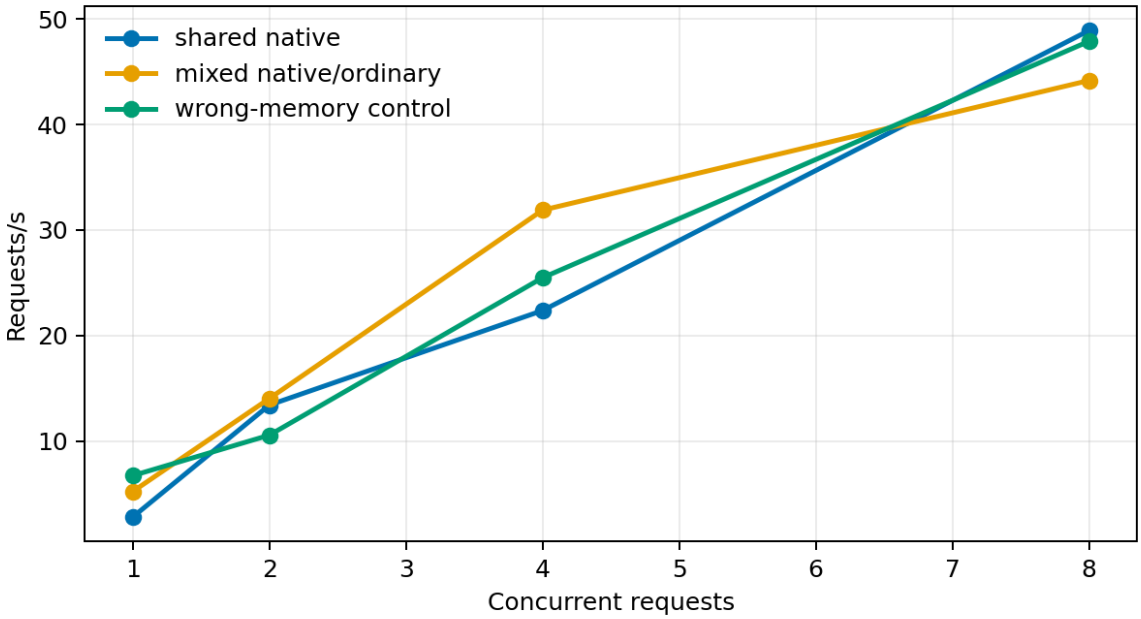

Figure 7: CUDA connector throughput and request-isolation controls. These are finite offline batches, not online arrival-process tail measurements.

eight-request V1 batches. CUDA now has a native-transfer connector candidate, but its source-length slots remain scheduler-visible. Remaining boundaries are an online arrival process, detached CUDA pages, and LMCache. The shared residency manager implements deterministic prefetch, pinning, sharing, and four eviction policies, but its pressure sweep is controlled simulation rather than vLLM scheduler telemetry.

A reproducible platform audit separates those boundaries from implementation status. vLLM-Metal 0.28.0 is available on the M5 and the native/APC path above is measured there. CUDA E0 is now measured on an RTX 5060 Laptop GPU; vLLM 0.28.0 requires its V1-runner fallback on the tested WSL configuration because the V2 runner's UVA allocation fails at startup. CUDA-native PRA page substitution is now measured through the prefix-shaped connector candidate, but detached attachment and LMCache remain unimplemented. HBM decomposition, PCIe/NVLink transfer, asynchronous promotion, and native CUDA tails remain explicitly unmeasured rather than inferred from Metal, E0, or completion time.

9 Falsification and Limitations

The central thesis fails if an optimized E0/G10 integration matches an engine-aware implementation at equal quality across memory pressure, concurrency, and reuse. The present data make that falsifier stronger: the black-box baseline already preserves quality, uses 76.9% fewer visible prompt tokens than full context, and coexists with APC. A native implementation must improve resident memory, transfer, TTFT, or throughput beyond this baseline.

The current study uses four 4-bit checkpoints from three model families, one Apple host, and one frozen 60-example natural-QA cohort per checkpoint. The cold requests are order-sensitive; concurrency is measured through the offline V1 engine rather than an HTTP server, and no arrival-rate, tail-latency, or HBM-pressure sweep was run. The live bridge requires page alignment, one K/V cache group, all consumer layers, and unquantized selected K/V. Apple unified memory also differs from the CUDA/LMCache environment for which many connector mechanisms were designed. The matched cohort establishes representation parity with frozen evidence; it does not establish that either condition answers every question correctly. Offline V1 also withholds per-token arrivals, so its TTFT and ITL columns remain unavailable. Selection-specific APC salts also mean that a changed PRA selection cannot reuse an APC namespace populated by a different selection, even when their visible conversation prefix is equal. The CUDA candidate adds one BF16 TinyLlama checkpoint and 60 natural examples, but runs one request at a time, serializes each source to host files, and does not report TTFT, ITL, transfer bytes, or K/V-only HBM. Its placeholders hide source content from model evaluation but retain source-length scheduler occupancy. It

is therefore neither a detached-page result nor LMCache evidence.

10 Related Work

vLLM introduced PagedAttention and continuous batching to reduce KV fragmentation and improve serving throughput [1]. Prefix caching reuses exact token histories; KV connectors and offload systems extend physical residency. Retrieval-augmented generation and sparse attention reduce the logical context presented to a model, but selected text is not equivalent to native non-prefix K/V. PRA combines explicit logical identity, retrieval, and native detail activation. The contribution here is the boundary between that logical mapping and vLLM's physical mapping.

11 Conclusion

Paper 6 establishes a reproducible vLLM-Metal baseline and a control plane that cannot accidentally overclaim native execution. Selected text recovers the fixed answer with 76.9% fewer visible tokens than full context and preserves APC. That success raises, rather than lowers, the bar for deep integration. The selected-K/V path is now verified at kernel level and in live V1 generation, including wrong-memory causality and post-request isolation. Native pages also coexist with APC and preserve request isolation through concurrency eight when APC identity includes the hidden selection. PRA-vLLM is therefore an engine-native controlled prototype on Metal with matched E0/E2 quality across Qwen, Llama, and Gemma checkpoints. CUDA E0 is reproduced on current NVIDIA hardware, and the semantic connector candidate preserves 60/60 natural outputs plus all causal controls. Its CUDA concurrency sweep preserves 45/45 request-scoped outcomes through concurrency eight with zero forbidden leaks and reaches 48.9 shared-native requests/s. Its source-length placeholder slots and inconsistent completion advantage prevent promotion to detached CUDA E2. The next decisive step is scheduler-invisible page attachment or an official connector mode that decouples external K/V from prompt-prefix accounting, followed by LMCache, matched online tails, HBM, transfer, and continuous-batching measurements before broader economic claims.

References

[1] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of SOSP*, 2023.